


```

        break;
    case 4: traverse();
        break;
    case 5: exit(0);
    default: printf("Invalid choice\n");
    }
}
return 0;
}

```

/* Create Linked List */

```

void create() {
    int n, i, value;
    struct node *temp, *newnode;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++) {
        newnode = (struct node *)malloc(sizeof(struct node));
        printf("Enter data: ");
        scanf("%d", &value);

        newnode->data = value;
        newnode->next = NULL;

        if (head == NULL) {
            head = newnode;
        } else {
            temp = head;
            while (temp->next != NULL)
                temp = temp->next;
            temp->next = newnode;
        }
    }
}

```

/* Insert at End */

```

void insert() {
    int value;

```

```

struct node *newnode, *temp;

newnode = (struct node *)malloc(sizeof(struct node));
printf("Enter value to insert: ");
scanf("%d", &value);

newnode->data = value;
newnode->next = NULL;

if (head == NULL) {
    head = newnode;
} else {
    temp = head;
    while (temp->next != NULL)
        temp = temp->next;
    temp->next = newnode;
}
}

/* Delete a Node */
void deletee() {
    int value;
    struct node *temp, *prev;

    if (head == NULL) {
        printf("List is empty\n");
        return;
    }

    printf("Enter value to delete: ");
    scanf("%d", &value);

    temp = head;

    if (temp->data == value) {
        head = temp->next;
        free(temp);
        printf("Node deleted\n");
        return;
    }
}

```

```

while (temp != NULL && temp->data != value) {
    prev = temp;
    temp = temp->next;
}

if (temp == NULL) {
    printf("Value not found\n");
} else {
    prev->next = temp->next;
    free(temp);
    printf("Node deleted\n");
}
}
/* Traverse the List */
void traverse() {
    struct node *temp;

    if (head == NULL) {
        printf("List is empty\n");
        return;
    }

    temp = head;
    printf("Linked List: ");
    while (temp != NULL) {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");}

```

Write a program that uses functions to perform the following operations on doubly linked list: i. Creation ii. Insertion iii. Deletion iv. Traversal

```
#include <stdio.h>
#include <stdlib.h>

/* Definition of node */
struct node {
    int data;
    struct node *prev;
    struct node *next;
};

struct node *head = NULL;

/* Function Prototypes */
void create();
void insert();
void deletee();
void traverse();

/* Main Function */
int main() {
    int choice;

    while (1) {
        printf("\n--- Doubly Linked List Menu ---\n");
        printf("1. Create\n");
        printf("2. Insert\n");
        printf("3. Deletee\n");
        printf("4. Traverse\n");
        printf("5. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1: create();
                    break;
            case 2: insert();
                    break;
```

```

        case 3: deletee();
            break;
        case 4: traverse();
            break;
        case 5: exit(0);
        default: printf("Invalid choice\n");
    }
}
return 0;
}

```

/* Create Doubly Linked List */

```

void create() {
    int n, i, value;
    struct node *temp, *newnode;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++) {
        newnode = (struct node *)malloc(sizeof(struct node));
        printf("Enter data: ");
        scanf("%d", &value);

        newnode->data = value;
        newnode->prev = NULL;
        newnode->next = NULL;

        if (head == NULL) {
            head = newnode;
        } else {
            temp = head;
            while (temp->next != NULL)
                temp = temp->next;

            temp->next = newnode;
            newnode->prev = temp;
        }
    }
}

```

```

/* Insert at End */
void insert() {
    int value;
    struct node *newnode, *temp;

    newnode = (struct node *)malloc(sizeof(struct node));
    printf("Enter value to insert: ");
    scanf("%d", &value);

    newnode->data = value;
    newnode->prev = NULL;
    newnode->next = NULL;

    if (head == NULL) {
        head = newnode;
    } else {
        temp = head;
        while (temp->next != NULL)
            temp = temp->next;

        temp->next = newnode;
        newnode->prev = temp;
    }
}

```

```

/* Delete a Node by Value */
void deletee() {
    int value;
    struct node *temp;

    if (head == NULL) {
        printf("List is empty\n");
        return;
    }

    printf("Enter value to delete: ");
    scanf("%d", &value);

    temp = head;

```

```

/* If first node is to be deleted */
if (temp->data == value) {
    head = temp->next;
    if (head != NULL)
        head->prev = NULL;
    free(temp);
    printf("Node deleted\n");
    return;
}

while (temp != NULL && temp->data != value)
    temp = temp->next;

if (temp == NULL) {
    printf("Value not found\n");
} else {
    temp->prev->next = temp->next;
    if (temp->next != NULL)
        temp->next->prev = temp->prev;
    free(temp);
    printf("Node deleted\n");
}
}

/* Traverse Forward and Backward */
void traverse() {
    struct node *temp;

    if (head == NULL) {
        printf("List is empty\n");
        return;
    }

    printf("Forward Traversal: ");
    temp = head;
    while (temp->next != NULL) {
        printf("%d <-> ", temp->data);
        temp = temp->next;
    }
}

```



```

printf("%d\n", temp->data);

printf("Backward Traversal: ");
while (temp != NULL) {
    printf("%d <-> ", temp->data);
    temp = temp->prev;
}
printf("NULL\n");
}

```

Write a program that uses functions to perform the following operations on circular linked i.
 Creation ii. Insertion iii. Deletion iv. Traversal

```

#include <stdio.h>
#include <stdlib.h>

/* Node structure */
struct node {
    int data;
    struct node *next;
};

struct node *head = NULL;

/* Function declarations */
void create();
void insert();
void delete();
void traverse();

/* Main function */
int main() {
    int choice;

    while (1) {
        printf("\n--- Circular Linked List Menu ---\n");
        printf("1. Create\n");
        printf("2. Insert\n");
        printf("3. Delete\n");
        printf("4. Traverse\n");
    }
}

```

```

printf("5. Exit\n");
printf("Enter your choice: ");
scanf("%d", &choice);

switch (choice) {
    case 1: create(); break;
    case 2: insert(); break;
    case 3: delete(); break;
    case 4: traverse(); break;
    case 5: exit(0);
    default: printf("Invalid choice\n");
}
}
return 0;
}

/* Create circular linked list */
void create() {
    int n, i, val;
    struct node *newnode, *temp;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++) {
        newnode = (struct node*)malloc(sizeof(struct node));
        printf("Enter data: ");
        scanf("%d", &val);

        newnode->data = val;

        if (head == NULL) {
            head = newnode;
            newnode->next = head;
        } else {
            temp = head;
            while (temp->next != head)
                temp = temp->next;

            temp->next = newnode;
        }
    }
}

```

```

        newnode->next = head;
    }
}

```

/* Insert at end */

```

void insert() {
    int val;
    struct node *newnode, *temp;

    newnode = (struct node*)malloc(sizeof(struct node));
    printf("Enter value to insert: ");
    scanf("%d", &val);

    newnode->data = val;

    if (head == NULL) {
        head = newnode;
        newnode->next = head;
    } else {
        temp = head;
        while (temp->next != head)
            temp = temp->next;

        temp->next = newnode;
        newnode->next = head;
    }
}

```

/* Delete node by value */

```

void delete() {
    int val;
    struct node *temp, *prev;

    if (head == NULL) {
        printf("List is empty\n");
        return;
    }

    printf("Enter value to delete: ");

```

```

scanf("%d", &val);

temp = head;
prev = NULL;

/* If head node is to be deleted */
if (temp->data == val) {
    if (temp->next == head) {
        free(temp);
        head = NULL;
        printf("Node deleted\n");
        return;
    }
    prev = head;
    while (prev->next != head)
        prev = prev->next;

    prev->next = temp->next;
    head = temp->next;
    free(temp);
    printf("Node deleted\n");
    return;
}

prev = temp;
temp = temp->next;

while (temp != head && temp->data != val) {
    prev = temp;
    temp = temp->next;
}

if (temp == head) {
    printf("Value not found\n");
} else {
    prev->next = temp->next;
    free(temp);
    printf("Node deleted\n");
}
}

```

```

/* Traverse circular linked list */
void traverse() {
    struct node *temp;

    if (head == NULL) {
        printf("List is empty\n");
        return;
    }

    printf("Circular Linked List: ");
    temp = head;
    do {
        printf("%d -> ", temp->data);
        temp = temp->next;
    } while (temp != head);
    printf("(back to head)\n");
}

```

Write a program that implements stack (its operations) using i. Arrays

```

#include <stdio.h>
#include <stdlib.h>

```

```

#define MAX 5

```

```

int stack[MAX];
int top = -1;

```

```

/* Function Prototypes */

```

```

void push();
void pop();
void peek();
void traverse();

```

```

/* Main Function */

```

```

int main()
{

```

```
int choice;
```

```
while(1)
```

```
{
```

```
    printf("\n--- Stack Menu ---\n");
```

```
    printf("1. Push\n");
```

```
    printf("2. Pop\n");
```

```
    printf("3. Peek\n");
```

```
    printf("4. Traverse\n");
```

```
    printf("5. Exit\n");
```

```
    printf("Enter your choice: ");
```

```
    scanf("%d",&choice);
```

```
switch(choice)
```

```
{
```

```
    case 1: push();
```

```
        break;
```

```
    case 2: pop();
```

```
        break;
```

```
    case 3: peek();
```

```
        break;
```

```
    case 4: traverse();
```

```
        break;
```

```
    case 5: exit(0);
```

```
    default: printf("Invalid choice\n");
```

```
}
```

```
}
```

```
return 0;
```

```
}
```

```
/* Push Operation */
```

```
void push()
```

```

{
    int value;

    if(top == MAX-1)
    {
        printf("Stack Overflow \n");
        return;
    }

    printf("Enter value to push: ");
    scanf("%d",&value);

    top = top + 1;
    stack[top] = value;

    printf("Element inserted\n");
}

/* Pop Operation */

void pop()
{
    if(top == -1)
    {
        printf("Stack Underflow\n");
        return;
    }

    printf("Deleted element is %d\n",stack[top]);
    top = top - 1;
}

/* Peek Operation */

void peek()
{
    if(top == -1)
    {
        printf("Stack is empty\n");
        return;
    }

```

```

    }

    printf("Top element is %d\n",stack[top]);
}

/* Traverse Stack */

void traverse()
{
    int i;

    if(top == -1)
    {
        printf("Stack is empty\n");
        return;
    }

    printf("Stack elements are:\n");

    for(i = top; i >= 0; i--)
    {
        printf("%d\n",stack[i]);
    }
}

```

Write a program that implement stack (its operations) using Linked List

```

#include <stdio.h>
#include <stdlib.h>

/* Definition of Node */
struct node
{
    int data;
    struct node *next;
};

struct node *top = NULL;

/* Function Prototypes */

```



```

        default: printf("Invalid choice\n");
    }
}

return 0;
}

/* Push Operation */

void push()
{
    int value;
    struct node *newnode;

    newnode = (struct node*)malloc(sizeof(struct node));

    printf("Enter value to push: ");
    scanf("%d",&value);

    newnode->data = value;
    newnode->next = top;
    top = newnode;

    printf("Element inserted\n");
}

/* Pop Operation */

void pop()
{
    struct node *temp;

    if(top == NULL)
    {
        printf("Stack Underflow\n");
        return;
    }

    temp = top;
    printf("Deleted element is %d\n", temp->data);

```

```

    top = top->next;
    free(temp);
}

/* Peek Operation */

void peek()
{
    if(top == NULL)
    {
        printf("Stack is empty\n");
        return;
    }

    printf("Top element is %d\n", top->data);
}

/* Traverse Stack */

void traverse()
{
    struct node *temp;

    if(top == NULL)
    {
        printf("Stack is empty\n");
        return;
    }

    temp = top;

    printf("Stack elements are:\n");

    while(temp != NULL)
    {
        printf("%d\n", temp->data);
        temp = temp->next;
    }
}

```

4. Write a program that implement stack (its operations) using

i. Arrays ii. Linked List

i. Arrays

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#define MAX 5
```

```
int stack[MAX];
```

```
int top = -1;
```

```
/* Function declarations */
```

```
void push();
```

```
void pop();
```

```
void peek();
```

```
void display();
```

```
int main()
```

```
{
```

```
    int choice;
```

```
    while(1)
```

```
{  
    printf("\n--- Stack Menu ---\n");  
    printf("1. Push\n");  
    printf("2. Pop\n");  
    printf("3. Peek\n");  
    printf("4. Display\n");  
    printf("5. Exit\n");  
    printf("Enter your choice: ");  
    scanf("%d", &choice);  
  
    switch(choice)  
    {  
        case 1: push(); break;  
        case 2: pop(); break;  
        case 3: peek(); break;  
        case 4: display(); break;  
        case 5: exit(0);  
        default: printf("Invalid choice\n");  
    }  
}  
  
/* Push operation */
```

```
void push()
{
    int item;

    if(top == MAX - 1)
    {
        printf("Stack Overflow\n");
        return;
    }

    printf("Enter element to push: ");
    scanf("%d", &item);

    top++;
    stack[top] = item;

    printf("Element pushed successfully\n");
}

/* Pop operation */
void pop()
{
    if(top == -1)
```

```
    {  
        printf("Stack Underflow\n");  
        return;  
    }  
  
    printf("Popped element: %d\n", stack[top]);  
    top--;  
}
```

/* Peek operation */

void peek()

```
{  
    if(top == -1)  
    {  
        printf("Stack is empty\n");  
        return;  
    }  
  
    printf("Top element: %d\n", stack[top]);  
}
```

/* Display operation */

void display()

```
{  
  
    int i;  
  
    if(top == -1)  
    {  
        printf("Stack is empty\n");  
        return;  
    }  
  
    printf("Stack elements:\n");  
    for(i = top; i >= 0; i--)  
    {  
        printf("%d\n", stack[i]);  
    }  
}
```

OUTPUT:

1. Push
2. Pop
3. Peek
4. Display
5. Exit

Enter choice: 1

Enter element: 10

Enter choice: 3

Top element: 10

Enter choice: 1

Enter element: 20

Enter choice: 3

Top element: 20

Enter choice: 4

Stack elements:

20

10

Enter choice: 2

Deleted element: 20

Enter choice: 3

Top element: 10

Enter choice: 5

ii. Linked List

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct node
```

```
{
```

```
    int data;
```

```
    struct node *next;
```

```
};
```

```
struct node *top = NULL;
```

```
/* Function declarations */
```

```
void push();
```

```
void pop();
```

```
void peek();
```

```
void display();
```

```
int main()
```

```
{
```

```
    int choice;
```

```
    while(1)
```

```
    {
```

```
        printf("\n--- Stack Using Linked List ---\n");
```

```
        printf("1. Push\n");
```

```
printf("2. Pop\n");

printf("3. Peek\n");

printf("4. Display\n");

printf("5. Exit\n");

printf("Enter your choice: ");

scanf("%d", &choice);


switch(choice)

{

case 1: push(); break;

case 2: pop(); break;

case 3: peek(); break;

case 4: display(); break;

case 5: exit(0);

default: printf("Invalid choice\n");

}

}


/* Push operation */

void push()

{

    struct node *ptr;
```

```
int value;
```

```
ptr = (struct node*)malloc(sizeof(struct node));
```

```
if(ptr == NULL)
```

```
{
```

```
printf("Stack Overflow\n");
```

```
return;
```

```
}
```

```
printf("Enter element to push: ");
```

```
scanf("%d", &value);
```

```
ptr->data = value;
```

```
ptr->next = top;
```

```
top = ptr;
```

```
printf("Element pushed successfully\n");
```

```
}
```

```
/* Pop operation */
```

```
void pop()
```

```
{
```

```
struct node *temp;
```

```
    if(top == NULL)

    {

        printf("Stack Underflow\n");

        return;

    }


    temp = top;

    printf("Popped element: %d\n", temp->data);

    top = top->next;

    free(temp);

}


/* Peek operation */

void peek()

{

    if(top == NULL)

    {

        printf("Stack is empty\n");

        return;

    }


    printf("Top element: %d\n", top->data);
```

```
}

/* Display operation */
void display()
{
    struct node *temp;

    if(top == NULL)
    {
        printf("Stack is empty\n");
        return;
    }

    temp = top;
    printf("Stack elements:\n");
    while(temp != NULL)
    {
        printf("%d\n", temp->data);
        temp = temp->next;
    }
}
```

OUTPUT:

1. Push

2. Pop

3. Display

4. Exit

Enter choice: 1

Enter element: 10

Enter choice: 1

Enter element: 20

Enter choice: 3

Stack elements:

20

10

Enter choice: 2

Deleted element: 20

Enter choice: 4

5. Write a program that implement Queue (its operations) using

i. Arrays ii. Linked List

i. Arrays

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#define MAX 5
```

```
int queue[MAX];
```

```
int front = -1, rear = -1;
```

```
/* Function declarations */
```

```
void enqueue();
```

```
void dequeue();
```

```
void peek();
```

```
void display();
```

```
int main()
```

```
{
```

```
    int choice;
```

```
    while(1)
```

```
    {
```



```

printf("\n--- Queue Menu ---\n");

printf("1. Enqueue(insert)\n");

printf("2. Dequeue(delete)\n");

printf("3. Peek\n");

printf("4. Display\n");

printf("5. Exit\n");

printf("Enter your choice: ");

scanf("%d", &choice);


switch(choice)
{
case 1: enqueue(); break;

case 2: dequeue(); break;

case 3: peek(); break;

case 4: display(); break;

case 5: exit(0);

default: printf("Invalid choice\n");

}

}

/* Enqueue operation */

void enqueue()

```

```
{  
  
    int item;  
  
    if(rear == MAX - 1)  
    {  
        printf("Queue Overflow\n");  
        return;  
    }  
  
    printf("Enter element to insert: ");  
    scanf("%d", &item);  
  
    if(front == -1)  
        front = 0;  
  
    rear++;  
    queue[rear] = item;  
  
    printf("Element inserted successfully\n");  
}  
  
/* Dequeue operation */  
void dequeue()
```

```
{  
  
    if(front == -1 || front > rear)  
  
    {  
  
        printf("Queue Underflow\n");  
  
        return;  
  
    }  
  
  
    printf("Deleted element: %d\n", queue[front]);  
  
    front++;  
  
  
    if(front > rear)  
  
    {  
  
        front = rear = -1;  
  
    }  
  
}
```

/* Peek operation */

void peek()

```
{  
  
    if(front == -1)  
  
    {  
  
        printf("Queue is empty\n");  
  
        return;  
  
    }  
  
}
```

```
    }

    printf("Front element: %d\n", queue[front]);
}

/* Display operation */
void display()
{
    int i;

    if(front == -1)
    {
        printf("Queue is empty\n");
        return;
    }

    printf("Queue elements:\n");
    for(i = front; i <= rear; i++)
    {
        printf("%d ", queue[i]);
    }

    printf("\n");
}
```

OUTPUT:

1. Enqueue

2. Dequeue

3. Display

4. Exit

Enter choice: 1

Enter element: 10

Enter choice: 1

Enter element: 20

Enter choice: 3

Queue elements: 10 20

Enter choice: 2

Deleted element: 10

Enter choice: 4

ii. Linked List

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct node
```

```
{
```

```
    int data;
```

```
    struct node *next;
```

```
};
```

```
struct node *front = NULL, *rear = NULL;
```

```
/* Function declarations */
```

```
void enqueue();
```

```
void dequeue();
```

```
void peek();
```

```
void display();
```

```
int main()
```

```
{  
  
    int choice;  
  
  
    while(1)  
    {  
  
        printf("\n--- Queue Using Linked List ---\n");  
printf("1. Enqueue\n");  
        printf("2. Dequeue\n");  
        printf("3. Peek\n");  
        printf("4. Display\n");  
        printf("5. Exit\n");  
  
        printf("Enter your choice: ");  
  
        scanf("%d", &choice);  
  
  
        switch(choice)  
        {  
  
            case 1: enqueue(); break;  
            case 2: dequeue(); break;  
            case 3: peek(); break;  
            case 4: display(); break;  
            case 5: exit(0);  
  
            default: printf("Invalid choice\n");  
        }  
    }  
}
```

```

    }
}

/* Enqueue operation */
void enqueue()
{
    struct node *ptr;
    int value;

    ptr = (struct node*)malloc(sizeof(struct node));
    if(ptr == NULL)
    {
        printf("Queue Overflow\n");
        return;
    }

    printf("Enter element to insert: ");
    scanf("%d", &value);

    ptr->data = value;
    ptr->next = NULL;

    if(rear == NULL)

```



```
{  
    front = rear = ptr;  
}  
  
else  
  
{  
    rear->next = ptr;  
    rear = ptr;  
}  
  
printf("Element inserted successfully\n");  
}
```

/* Dequeue operation */

void dequeue()

```
{  
  
    struct node *temp;  
  
    if(front == NULL)  
    {  
        printf("Queue Underflow\n");  
        return;  
    }  
}
```

```
temp = front;

printf("Deleted element: %d\n", temp->data);

front = front->next;


if(front == NULL)

rear = NULL;


free(temp);
}
```

```
/* Peek operation */
```

```
void peek()
```

```
{

    if(front == NULL)

    {

        printf("Queue is empty\n");

        return;

    }


    printf("Front element: %d\n", front->data);

}
```

```
/* Display operation */
```

```
void display()
{
    struct node *temp;

    if(front == NULL)
    {
        printf("Queue is empty\n");
        return;
    }

    temp = front;
    printf("Queue elements: ");
    while(temp != NULL)
    {
        printf("%d ", temp->data);
        temp = temp->next;
    }
    printf("\n");
}
```

OUTPUT:

1. Enqueue
2. Dequeue
3. Display

4. Exit

Enter choice: 1

Enter element: 10

Enter choice: 1

Enter element: 20

Enter choice: 3

Queue elements: 10 20

Enter choice: 2

Deleted element: 10

Enter choice: 4

